

Enci/ VENTAS

Arquitectura Detallada y Flujos

ANEXO B

Proyecto Enci / Enci Ventas

Aplicación de Automatización de Fuerza de Ventas

Equipo de Desarrollo: Grupo PLM - Universidad Tecnológica Metropolitana

Líder: Joshua Flores

Integrantes: Falon Berrios, Carolaine Palacios y Eliseo Poblete

Versión 1.0

Julio 2026

Tabla de Contenidos

1. Introducción y propósito
2. Resumen de arquitectura
3. Principios técnicos de diseño
4. Flujo de autenticación y sesión
5. Autorización por rol, plataforma y propiedad
6. Componentes backend FastAPI
7. Pipeline de request, middlewares y errores
8. Headers de seguridad y exposición de documentación
9. Readiness y validación de ambiente
10. Persistencia, Firestore y modelo de acceso
11. Flujos funcionales del MVP web
12. Módulo IA RAG con DeepSeek
13. Plataforma web, PWA vendedor y app Flutter
14. Despliegue, configuración e infraestructura
15. Riesgos arquitectónicos aceptados
16. Criterios de aceptación del anexo
17. Recomendaciones de continuidad técnica

1. Introducción y propósito

Este anexo documenta la arquitectura detallada y los flujos principales del sistema Enci / Enci Ventas. Su finalidad es entregar una visión técnica ordenada del funcionamiento interno del MVP web avanzado, sus componentes backend y frontend, su integración con Firebase, Firestore, RAG, plataforma móvil y condiciones de despliegue.

El documento está preparado para revisión técnica y recepción formal por parte del cliente. Explica cómo circulan las solicitudes desde el usuario hasta la API, cómo se valida la identidad, cómo se aplican roles, cómo se persisten los datos y qué controles transversales protegen el sistema.

Alcance del anexo
Este documento describe la arquitectura comprometida para la entrega técnica. No sustituye los manuales de instalación, despliegue, variables, usuario, soporte, seguridad o API. Su función es explicar la lógica estructural y los flujos que sostienen dichos documentos.

1.1 Responsables del documento

Campo	Detalle
Proyecto	Enci / Enci Ventas
Tipo de sistema	MiniCRM comercial y plataforma de automatización de fuerza de ventas
Equipo responsable	Grupo PLM - Universidad Tecnológica Metropolitana
Líder	Joshua Flores
Integrantes	Falon Berrios, Carolaine Palacios y Eliseo Poblete
Versión	1.0
Fecha	Julio 2026

2. Resumen de arquitectura

La solución se organiza como una arquitectura web con backend API, frontend SPA, autenticación centralizada, persistencia administrada y componentes complementarios para analítica, RAG y experiencia móvil. El backend concentra la autorización y las reglas de negocio; el frontend consume datos por API y organiza la experiencia visual por rol.

Capa	Tecnología	Responsabilidad arquitectónica
Frontend web	React + Vite	Interfaz SPA del MVP comercial, rutas protegidas, pantallas operativas y consumo de API.
Autenticación	Firebase Auth + Google SSO	Inicio de sesión, emisión de ID Token y administración de identidad base.
Backend	FastAPI + Python	API REST, validaciones, permisos, reglas de negocio, Swagger controlado y readiness.

Capa	Tecnología	Responsabilidad arquitectónica
Autorización	Firebase Admin SDK + Firestore users	Verificación de token, rol, estado activo, plataforma y propiedad de registros.
Base de datos	Cloud Firestore	Persistencia de usuarios, clientes, interacciones, oportunidades, propuestas, documentos RAG y conversaciones.
RAG parcial	DeepSeek + Chroma	Chat documental y contextual mediante chunks, embeddings, fuentes y proveedor LLM.
PWA vendedor	React/Vite	Experiencia móvil web instalable relacionada al equipo comercial.
App Flutter	Flutter + Firebase	Entregable nativo Android parcial, desarrollado fuera del monorepo y sujeto a aceptación parcial.
Deploy previsto	Cloud Run + Vercel	Backend en GCP y frontend publicado de forma independiente.

2.1 Diagrama lógico general

Usuario comercial

- > Frontend React/Vite o experiencia móvil
- > Firebase Auth / Google SSO
- > ID Token Firebase
- > Authorization: Bearer <token>
- > FastAPI
- > Firebase Admin verify_token
- > Firestore users/{uid}
- > rol + activo + acceso webApp/appMovil
- > servicio de negocio
- > Firestore / Chroma / proveedor externo según módulo
- > respuesta API
- > interfaz de usuario

3. Principios técnicos de diseño

La arquitectura del proyecto se basa en separación de responsabilidades, autorización centralizada y trazabilidad técnica. Cada capa cumple una función específica y las decisiones sensibles quedan concentradas en backend.

Principio	Aplicación en el sistema	Criterio de control
Backend como autoridad	La API valida token, rol, estado, plataforma y propiedad antes de ejecutar operaciones.	El frontend no decide permisos reales.
Cliente sin escritura directa	Firestore queda protegido mediante reglas deny-all para acceso directo desde cliente.	Toda operación pasa por API.
Separación de capas	Routers, modelos y servicios separan entrada HTTP, validación y lógica de negocio.	Facilita pruebas y mantención.

Principio	Aplicación en el sistema	Criterio de control
Configuración por ambiente	Variables de entorno controlan Firebase, CORS, RAG, DeepSeek y almacenamiento.	No versionar secretos.
UAT controlado	Readiness, healthcheck, Swagger y smoke tests permiten revisar salida a ambiente objetivo.	Go-live depende de configuración real.
Alcance parcial explícito	RAG y Flutter se documentan como componentes adicionales parciales.	No generan obligación de extensión futura.

4. Flujo de autenticación y sesión

El flujo de autenticación comienza en la interfaz, pero la validación final ocurre en backend. Firebase Auth entrega un ID Token al cliente; ese token se adjunta en cada solicitud protegida y FastAPI lo valida mediante Firebase Admin antes de consultar el perfil operativo en Firestore.

4.1 Secuencia de autenticación

Paso	Actor	Acción	Resultado esperado
1	Usuario	Accede a la aplicación y selecciona inicio con Google.	Se inicia flujo Google SSO.
2	Firebase Auth	Autentica identidad y emite ID Token.	El frontend recibe credencial temporal.
3	Frontend	Adjunta Authorization: Bearer <firebase_id_token> en cada request.	La API recibe identidad verificable.
4	FastAPI	Ejecuta verify_token con Firebase Admin.	Token válido o error 401.
5	Backend	Consulta users/{uid} en Firestore.	Obtiene rol, nombre, activo, webApp y appMovil.
6	Backend	Valida usuario activo y acceso a plataforma.	Permite continuar o responde 403.
7	Servicio	Ejecuta regla de negocio según rol y propiedad.	Retorna datos permitidos.
8	Frontend	Renderiza respuesta o estado de error.	Usuario ve datos, 403, sesión expirada o pantalla correspondiente.

4.2 Decisiones del flujo de sesión

- Sin token o con token inválido, el backend responde 401 y no ejecuta lógica de negocio.
- Con token válido pero usuario no registrado, inactivo o sin rol habilitado, el backend responde 403.

- Con usuario activo, el backend normaliza rol y verifica acceso a la plataforma solicitada.
- La sesión del frontend no debe persistir credenciales sensibles fuera de los mecanismos controlados de Firebase y la aplicación.
- El cierre de sesión y la expiración deben redirigir al login y limpiar el estado de usuario local.

5. Autorización por rol, plataforma y propiedad

La autorización se aplica en tres niveles: rol del usuario, plataforma de acceso y propiedad del registro. Esta combinación evita que un usuario vea o modifique información fuera de su alcance, incluso si manipula rutas, IDs o payloads desde el navegador.

Nivel	Dato evaluado	Regla
Identidad	uid Firebase	Debe existir un usuario autenticado y verificable.
Estado	activo	Usuarios inactivos quedan bloqueados aunque su token sea válido.
Rol	admin, supervisor, vendedor	Cada endpoint exige un nivel mínimo de rol.
Plataforma	X-Enci-Client	web se traduce a webApp; mobile/app se traduce a appMovil.
Propiedad	vendedorUid / createdBy / owner	Vendedor opera solo datos propios; supervisor y admin tienen visibilidad superior según endpoint.

5.1 Matriz de roles

Rol	Permisos generales	Restricción crítica
Vendedor	Gestiona clientes propios, interacciones, oportunidades, propuestas y consultas RAG de su cuenta.	No debe acceder a registros de otros vendedores.
Supervisor	Accede a dashboards, reportes, equipo y datos consolidados según módulo.	No reemplaza necesariamente a admin para cambios sensibles.
Admin	Gestiona usuarios, roles, importación, documentos RAG y administración general.	Debe operar con trazabilidad y cuidado sobre cambios críticos.
Sin acceso	No posee rol operativo habilitado.	Debe quedar bloqueado en endpoints protegidos.

6. Componentes backend FastAPI

La aplicación FastAPI se inicializa con configuración centralizada, routers por dominio, middlewares de seguridad, handlers de error y endpoints de verificación. En producción, la exposición de Swagger/OpenAPI debe permanecer controlada para reducir superficie

pública.

Componente	Función arquitectónica
main.py	Inicializa app, registra CORS, rate limit, request size limit, handlers, headers, routers, health, readiness y /me.
core/config.py	Centraliza settings de ambiente, CORS, Firebase, DeepSeek, Chroma, GCS, límites RAG y validaciones de seguridad.
core/auth.py	Verifica token, consulta usuario, normaliza rol, valida activo, webApp/appMovil y aplica require_role.
core/readiness.py	Construye reporte no sensible para condiciones mínimas de UAT/go-live.
core/rate_limit.py	Limita solicitudes por IP y tamaño de request antes del procesamiento principal.
core/errors.py	Estandariza errores HTTP, validación, Google API y excepciones no controladas.
api/auth.py	Login, registro y sincronización de usuario autenticado.
api/clientes.py	Operaciones CRUD y baja lógica sobre clientes.
api/comercial.py	Interacciones, oportunidades y propuestas.
api/dashboard.py	Métricas comerciales, embudo, forecast, equipo y agregados.
api/competencia.py	Información competitiva y análisis asociado.
api/rag.py	Chat RAG, conversaciones, documentos, carga, reindexación y listado.
api/users.py	Gestión de usuarios, roles, accesos y estados.
docs.py	Documentación Swagger/OpenAPI bajo condiciones controladas.

6.1 Routers incluidos

```
auth_router
clientes_router
comercial_router
competencia_router
dashboard_router
rag_router
users_router
docs_router
```

7. Pipeline de request, middlewares y errores

Cada solicitud atraviesa controles transversales antes de llegar a la lógica de negocio. Esto reduce abuso, payloads excesivos, errores no normalizados y respuestas inseguras.

Capa	Control	Finalidad
Entrada HTTP	CORS	Permitir solo orígenes configurados para el frontend web y apps autorizadas.

Capa	Control	Finalidad
Middleware	Rate limit en memoria	Limitar ráfagas básicas por IP durante el MVP.
Middleware	Request size limit	Rechazar payloads sobredimensionados antes de procesar contenido.
Middleware RAG	Límite específico de carga documental	Permitir documentos hasta el máximo configurado sin abrir el límite global.
Autenticación	HTTPBearer	Obtener token Bearer desde Authorization.
Autorización	require_role	Aplicar rol mínimo, usuario activo y acceso a plataforma.
Modelo	Pydantic	Validar payload, tipos, campos obligatorios y campos no permitidos.
Servicio	Regla de negocio	Aplicar propiedad, cálculos, persistencia y agregados.
Salida	Headers y errores	Responder con estados controlados, seguridad HTTP y mensajes no sensibles.

7.1 Manejo de errores

Tipo de error	Respuesta esperada	Uso
401 Unauthorized	Token inválido, ausente o expirado.	Protección de endpoints.
403 Forbidden	Usuario sin permiso, inactivo, sin rol o sin acceso a plataforma.	Autorización por rol/plataforma/estado.
404 Not Found	Recurso o ruta inexistente.	Rutas no válidas y recursos no encontrados.
422 Unprocessable Entity	Payload inválido o campos no permitidos.	Validación de modelos.
429 Too Many Requests	Límite excedido o cuota Firestore agotada según caso.	Rate limit y protección operativa.
503 Service Unavailable	Servicio de datos temporalmente no disponible.	Falla externa o Firestore inaccesible.

8. Headers de seguridad y exposición de documentación

El backend incorpora headers de seguridad para reducir riesgos comunes de navegador y exposición involuntaria. La documentación interactiva queda disponible bajo condiciones controladas y debe restringirse en producción real.

Header / política	Valor o criterio	Finalidad
X-Content-Type-Options	nosniff	Evitar interpretación insegura de MIME types.
X-Frame-Options	DENY	Bloquear carga en iframes no autorizados.
Referrer-Policy	no-referrer	Reducir filtración de URLs o parámetros.
Permissions-Policy	camera=(), microphone=(), geolocation=()	Deshabilitar permisos de navegador no requeridos.
Content-Security-Policy	Restrictiva por defecto	Evitar carga de recursos externos no autorizados.
CSP en /docs	Permite recursos controlados de unpkg.com y Firebase	Habilitar Swagger UI y consola de pruebas sin abrir toda la aplicación.

8.1 Exposición de Swagger / OpenAPI

- En ambiente de desarrollo o UAT controlado, Swagger puede usarse para validación técnica, demostraciones y pruebas de endpoints protegidos.
- En producción, /docs y /openapi.json deben permanecer deshabilitados o protegidos conforme a configuración del backend.
- Las pruebas en Swagger deben usar token Firebase real y esquema HTTPBearer.
- La consola de pruebas no reemplaza UAT funcional con usuarios reales, pero permite validar contratos y respuestas API.

9. Readiness y validación de ambiente

El endpoint GET /readiness entrega un reporte no sensible de condiciones mínimas de salida. Su objetivo es detectar errores de configuración antes de UAT o go-live, sin exponer secretos.

Check	Descripción	Criterio esperado
FIREBASE_PROJECT_ID	Confirma que existe proyecto Firebase configurado.	No debe ser placeholder.
Credenciales Firebase Admin	Valida que existe mecanismo de autenticación backend.	Debe estar configurado por archivo o variable segura.
CORS	Confirma orígenes permitidos.	Debe incluir dominios reales del frontend.
CORS desplegado	Bloquea localhost en ambientes uat/staging/production.	No usar 127.0.0.1 o localhost en salida real.
DeepSeek API key	Verifica disponibilidad del proveedor RAG.	Debe existir solo en backend si se usa RAG.
DeepSeek base URL	Confirma URL HTTPS del proveedor.	Debe usar HTTPS.

Check	Descripción	Criterio esperado
Modelo DeepSeek	Identifica modelo configurado.	Debe estar definido.
Dependencia openai	Verifica librería cliente para proveedor compatible.	Debe estar disponible en backend.
Role switcher temporal	Evita habilitar cambio temporal de rol en producción.	Debe ser false.

10. Persistencia, Firestore y modelo de acceso

La persistencia se realiza en Cloud Firestore mediante Firebase Admin SDK desde backend. El frontend no escribe directo en Firestore. Esta decisión mantiene la lógica de autorización y validación dentro de la API y evita que reglas del cliente se conviertan en el único control de seguridad.

10.1 Flujo de persistencia

Frontend autenticado -> FastAPI endpoint protegido -> get_current_user / require_role -> servicio de negocio -> validación de propiedad y payload -> Firestore mediante Firebase Admin SDK -> respuesta normalizada al frontend		
Colección / dominio	Uso probable	Control principal
users	Perfil operativo, rol, estado activo, accesos webApp/appMovil.	Consultado en cada flujo protegido.
clientes	Clientes comerciales y datos de contacto.	Visibilidad por vendedorUid y roles superiores.
interacciones	Registro de llamadas, correos, reuniones, visitas y otros contactos.	Vinculación a cliente visible.
oportunidades	Pipeline comercial, etapas e historial.	Propiedad del vendedor o visibilidad agregada.
propuestas	Propuestas vinculadas a oportunidad.	Cálculos server-side y relación con oportunidad permitida.
documentosRAG	Metadatos de documentos cargados para corpus.	Administración restringida a admin.
chatConversaciones	Historial de conversaciones RAG por usuario.	Vendedor ve historial propio; supervisor/admin consolidado según alcance.
audit_logs	Eventos críticos y cambios operativos.	Trazabilidad de acciones sensibles.

11. Flujos funcionales del MVP web

Los flujos funcionales comparten la misma estructura arquitectónica: el usuario opera desde el frontend, el frontend envía token, el backend valida identidad y permisos, los servicios aplican reglas de negocio y Firestore persiste o recupera información.

11.1 Flujo CRM de clientes

```
Usuario vendedor/admin
-> pantalla Clientes / Crear Cliente / Detalle Cliente
-> API /clientes
-> validación token + rol + activo
-> validación vendedorUid o rol superior
-> Firestore clientes
-> listado, detalle, creación, edición o baja lógica
```

11.2 Flujo comercial

Módulo	Entrada principal	Regla de negocio
Interacciones	Cliente, tipo, fecha, notas y responsable.	Solo se permite registrar sobre clientes visibles para el usuario.
Oportunidades	Cliente, etapa, monto, probabilidad, fecha y vendedor.	Debe existir cliente permitido y etapa válida.
Propuestas	Oportunidad, productos/servicios, descuento y estado.	El backend calcula descuento y total; no confiar en cálculo frontend.
Dashboard	Usuario, rol, periodo y filtros.	Vendedor ve métricas propias; supervisor/admin datos consolidados.
Usuarios	Rol, activo, accesos y datos operativos.	Cambios sensibles restringidos a admin.

11.3 Flujo dashboard

```
Usuario autenticado
-> Dashboard / Proyecciones / Equipo
-> API dashboard
-> rol vendedor/supervisor/admin
-> consultas agregadas a clientes, oportunidades, propuestas e interacciones
-> KPIs, embudo, forecast, rendimiento, alertas y matrices
```

12. Módulo IA RAG con DeepSeek

El módulo IA RAG se considera componente parcialmente entregado. Su arquitectura integra documentos cargados por administrador, procesamiento en chunks, índice Chroma persistido, proveedor DeepSeek y conversaciones asociadas a usuarios autenticados. La entrega cubre el estado actual del componente y no implica tuning, reentrenamiento, cambio de proveedor, evaluación avanzada de calidad ni SLA externo.

Elemento	Implementación	Control
Chat	POST /rag/chat	Requiere usuario con rol mínimo vendedor.
Pregunta	Sanitización y límite de caracteres	Evita entradas vacías o excesivas.
Contexto documental	Búsqueda de chunks similares en Chroma	Usa máximo de fragmentos configurado.
Contexto interno	Consulta Firestore cuando corresponde	Respetar rol y visibilidad del usuario.
Proveedor	DeepSeek mediante variable backend	API key no se expone en frontend.
Conversaciones	GET /rag/conversations	Historial propio o consolidado según rol.
Carga documental	POST /rag/documents/upload	Solo admin; valida formato y tamaño.
Reindexación	POST /rag/documents/reindex	Reconstruye índice desde documentos activos.
Listado documental	GET /rag/documents	Solo admin.

12.1 Flujo RAG

```

Usuario autenticado
-> Asistente Enci / Enci Chat
-> POST /rag/chat
-> rate limit por uid
-> sanitización de pregunta
-> búsqueda documental en Chroma
-> búsqueda de contexto interno en Firestore si aplica
-> selección de chunks
-> DeepSeek si existe contexto suficiente
-> respuesta con fuentes, conversación y diagnóstico técnico controlado
-> persistencia en chatConversaciones

```

Criterio de alcance RAG

El componente RAG se entrega en el estado actual. No incluye optimización de prompts, entrenamiento, migración a otro proveedor, carga masiva de corpus, evaluación formal de calidad ni garantía de disponibilidad del proveedor externo.

13. Plataforma web, PWA vendedor y app Flutter

La arquitectura del proyecto reconoce tres experiencias relacionadas: el frontend web principal, la PWA vendedor asociada al repositorio y la app nativa Flutter desarrollada fuera del monorepo. Estos componentes no deben confundirse en la evaluación técnica.

Componente	Ubicación / origen	Criterio de aceptación
Frontend web	frontend/ en monorepo	Entregable principal del MVP web avanzado.

Componente	Ubicación / origen	Criterio de aceptación
PWA vendedor	vendedor-app/ en monorepo	Experiencia móvil web instalable para vendedor; se valida según su propio build y variables.
App Flutter nativa	Desarrollada fuera del monorepo	Entregable parcial Android conforme a lo conversado con Max; su ausencia en GitHub no invalida su existencia.

13.1 Distinción crítica para recepción

App Flutter

La app nativa Flutter debe entenderse como componente externo y parcialmente realizado. No se evalúa por presencia dentro del repositorio GitHub Enci. Su aceptación corresponde al estado demostrado y al alcance acordado: UI, login Firebase, navegación y funciones parciales, sin garantía de cobertura total del backend ni publicación en tiendas.

14. Despliegue, configuración e infraestructura

El despliegue previsto separa backend y frontend. El backend puede desplegarse en Cloud Run u otro servicio compatible con contenedores Python/FastAPI; el frontend web y la PWA pueden publicarse en Vercel u otro hosting estático compatible con Vite. La correcta operación depende de variables, dominios, credenciales y CORS configurados de forma coherente.

Elemento	Condición de despliegue	Riesgo si falta
Backend API	Variables de entorno, credenciales Firebase Admin, CORS y puerto de servicio.	API no inicia, falla auth o bloquea frontend.
Frontend web	Variables VITE_*, dominio autorizado en Firebase y API pública HTTPS.	Login o llamadas API fallan.
PWA vendedor	Root y variables propias, proxy/API y Firebase Auth autorizados.	Experiencia móvil no consume backend.
Firestore	Proyecto correcto, índices necesarios, reglas deny-all y permisos service account.	Errores de consulta o exposición indebida.
RAG	DEEPSEEK_API_KEY, Chroma persistente, GCS o almacenamiento local según ambiente.	Chat sin proveedor, sin índice o sin documentos.
Dominios	Autorizados en Firebase Auth y CORS backend.	Bloqueo por origen no permitido.
Readiness	Debe revisarse antes de UAT/go-live.	Salida sin configuración mínima validada.

15. Riesgos arquitectónicos aceptados

Los siguientes riesgos están documentados como condiciones conocidas del MVP. No invalidan el cierre de Development, pero deben considerarse antes de una operación

productiva con usuarios reales, volumen alto o SLA formal.

Riesgo	Descripción	Tratamiento recomendado
Rate limit en memoria	No es distribuido entre múltiples instancias.	Migrar a Redis, API Gateway o control gestionado si se escala.
Índices Firestore	Algunas consultas pueden requerir índices adicionales con volumen real.	Monitorear errores de índice y crearlos según necesidad.
Backups	La configuración final depende del ambiente del cliente.	Activar políticas de respaldo y recuperación.
Monitoreo	Logs, métricas y alertas deben configurarse en infraestructura final.	Integrar Cloud Logging/Monitoring o equivalente.
RAG externo	DeepSeek depende de proveedor, cuota, API key y disponibilidad.	Definir política de uso, costos y fallback.
Chroma local	Índice local requiere persistencia y respaldo si se usa en producción.	Usar volumen persistente o servicio vectorial administrado.
Flutter parcial	No tiene cobertura total de backend ni publicación en tiendas dentro del alcance.	Tratar evolución como fase posterior.
Go-live	Depende de dominio, credenciales, reglas comerciales y UAT.	Separar cierre Development de salida productiva.

16. Criterios de aceptación del anexo

Este anexo se considera aceptable si permite comprender la arquitectura del sistema, explicar sus flujos principales, identificar controles críticos y distinguir los componentes principales, parciales y externos del proyecto.

Código	Criterio	Resultado esperado
CA-B01	Describe la arquitectura general de backend, frontend, Firebase, Firestore, RAG y móvil.	El lector entiende la composición técnica del sistema.
CA-B02	Documenta el flujo de autenticación con Google SSO, Firebase Auth, ID Token y Firebase Admin.	El proceso de login y validación queda claro.
CA-B03	Explica autorización por rol, plataforma y propiedad.	Se evita delegar seguridad al frontend.
CA-B04	Enumera routers, middlewares, headers, readiness y persistencia.	El traspaso técnico identifica puntos críticos del backend.
CA-B05	Incluye flujos funcionales de clientes, comercial, dashboard, usuarios y RAG.	La operación del MVP queda trazable.
CA-B06	Distingue PWA vendedor de app Flutter nativa externa parcial.	La ausencia de Flutter en GitHub no se interpreta como incumplimiento.
CA-B07	Registra riesgos arquitectónicos aceptados y recomendaciones futuras.	El cliente recibe una visión honesta del estado técnico.

17. Recomendaciones de continuidad técnica

- Mantener la autorización efectiva en backend y no trasladarla al frontend.
- Actualizar documentación cada vez que cambie un endpoint, variable, rol, flujo o condición de despliegue.
- Separar bugs UAT de cambios de alcance, especialmente en RAG, Flutter, ERP/SAP, tiendas móviles y soporte post-entrega.
- Configurar monitoreo, alertas, backups y observabilidad antes de salida productiva real.
- Formalizar credenciales, dominios, CORS, cuentas de prueba y usuarios reales antes de UAT.
- Definir estrategia de persistencia del índice RAG si el cliente decide operar el módulo en ambiente productivo.
- Tratar cualquier evolución de Flutter como nueva fase, especialmente integración completa de endpoints, publicación en tiendas, firma de APK/AAB, soporte de dispositivos y mantenimiento.

Cierre del anexo

La arquitectura documentada permite comprender el estado técnico comprometido para el proyecto Enci / Enci Ventas. La recepción debe distinguir entre MVP web cerrado para Development, componentes pendientes de UAT, IA RAG parcial y app Flutter nativa parcialmente realizada fuera del monorepo.